

INFORME DE DESARROLLO – PROYECTO #2

KANBAN TASKSCRIPT

Miguel Andrés Álvarez López 202406012 – Lenguajes Formales y de Programacion I

Problema #1 como diablos empiezo:

Para el proyecto, era muy similar al anterior solamente que implementando un parser en el lo cual era complicado al principio.

SOLUCION:

Como es un proyecto muy similar al proyecto 1, se utilizo como base al proyecto anterior trasladando todo a Widgets de QtCreator

```
void MainWindow::conectarUI(){
    connect(ui->btn_load, &QPushButton::clicked, this, &MainWindow::cargarArchivo);
    connect(ui->btn_analizar, &QPushButton::clicked, this, &MainWindow::analizarArchivo);
    connect(ui->btnReporte1, &QPushButton::clicked, this, &MainWindow::abrirReporte1);
    connect(ui->btnReporte2, &QPushButton::clicked, this, &MainWindow::abrirReporte2);
    connect(ui->btnReporte5, &QPushButton::clicked, this, &MainWindow::abrirReporte5);
    //connect funciona para vincular botones de la Interfaz grafica y que hagan una accioin
    //que se definio en el header de private slots

    for (auto *btn : {ui->btnReporte1, ui->btnReporte2, ui->btnReporte5}) {
        btn->setEnabled(false);
    }
    ui->btn_analizar->setEnabled(false);
}
```

REIMPLEMENTACION DE INTERFAZ POR MEDIO DE WIDGETS

Problema #2 Implementacion de Parser para Analisis Sintactico:

Para el proyecto, era muy similar al anterior incluso utilizando tokenizacion y similar la pregunta era como podía relacionar lo que ya tenia para algo nuevo. Donde comienzo?

SOLUCION:

En TOKENIZE se agrego una sintaxis para el parseo el cual registra el inicio, luego se llama a CONSUME PARA REALIZAR la produccion para cada parte del programa, el cual requiere que el TokenEsperado Sea el identico al actual que se esta recorriendo

```
//SINTAX PARSEO

this->tokens = tokens;
tokenActual = 0;
try {
    parsePrograma();//Inicio de parseo
    if(tokenActual < (int)tokens.size()){
        registrarError_S("ERROR,TOKENS SIN PROCESAR",linea,columna);
    }
} catch (exception e) {
    qWarning() << "Error en la lectura Sintactica" << e.what();
}

//!
qDebug() << "=== TOKENIZE END ===";
qDebug() << "Columnas registradas:" << columnStorage.size();

return tokens;//retorno
//!! DEBUGG
```

```
Token LexicalAnalyzer::Consume(TokenType await){
    //SI EL TOKEN ACTUAL ESTA UBICADO FUERA DE RANGO
    if(tokenActual >= (int)tokens.size()){
        registrarError_S("ERROR,TOKENS SIN PROCESAR",linea,columna);
        //retorna a la caja de tokens
        return Token{TokenType::DESCONOCIDO,"?",linea,columna};
    }

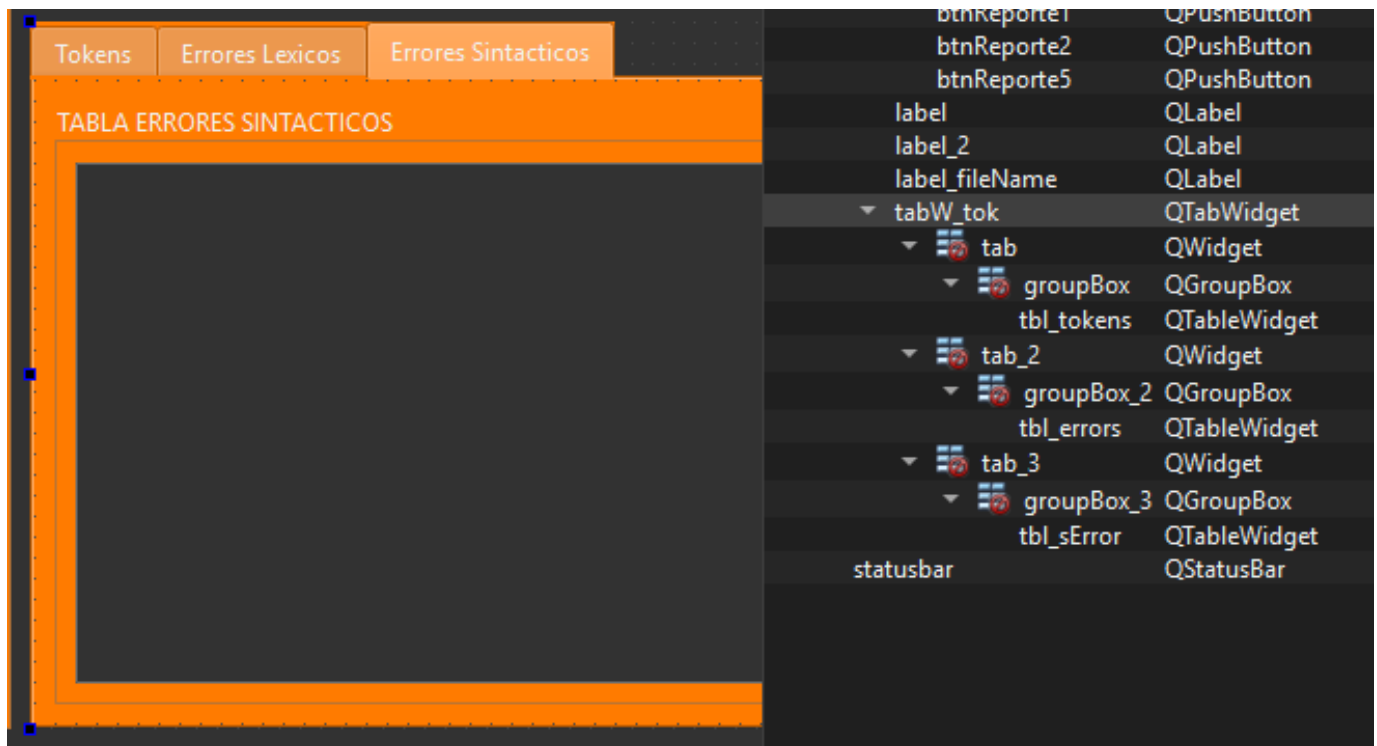
    Token t = tokens[tokenActual]; //Token actual lo tomara
    if(t.tipo == await){ //SI EN CASO DE QUE EL TIPO DE TOKEN SEA EL ESPERADO
        tokenActual++;
        return t; //RETORNA TOKEN ACTUAL SUMADO
    }else{
        registrarError_S(tokenTypeToString(await) + ", Token Diferente al esperado",t.linea,t.columna);
        return Token{TokenType::DESCONOCIDO,"",0,0};
    }
}
```

Problema #3 Implementar pestaña de Tablas

Para mantener una interfaz compacta se buscaba juntar todas las tablas en una sola pestaña la cual contendría las 3 tablas para tokens, errores léxicos y Sintacticos (Por separado) sabiendo que en el código anterior era escrito a código era muy confuso buscar una alternativa en Widgets

SOLUCION:

Por medio de Un QTabWidgets se logro implementar las 3 tablas por pestañas, además de ser mas flexible al tratar de acomodar las tablas y contenedors



Problema #4 Generacion de Reportes de Carga por persona y Errores Lexicos y Sintacticos

Para la generación de reportes se necesitaba buscar una forma de imprimirlos por PDF.

SOLUCION:

Como se utilizo el proyecto anterior como base no hubo mucha necesidad de cambiar la forma de como se generaban los reportes ,simplemente se modifiko la estructura para cada uno

Problema #5 Generacion de Reporte KanBan

Como el Unico Reporte que cambiaba su metodología de utilizar Tablas era el del Reporte KANBAN Tenia que buscar una alternativa parecida a la que era rrequerida en el pdf del proyecto 2, por lo cual tuve que buscar una opción para no utilizar tablas.

SOLUCION:

<DIV> funcionando como contenedores y que tuviera estilos según como estaba estructurado <Div> Contenedor -> <Div> -> carta de Tipo de Prioridad, cada uno de manera horizontal y con su propio estilo según el nivel de Prioridad

todas las tareas se registran en el tablero KANBAN

